```vhdl
-- ========================================
-- COMMON COMBINATIONAL CIRCUITS IN VHDL
-- ========================================

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

-- ========================================
-- 1. HALF ADDER
-- ========================================
entity half_adder is
    Port (
        A : in STD_LOGIC;
        B : in STD_LOGIC;
        Sum : out STD_LOGIC;
        Carry : out STD_LOGIC
    );
end half_adder;

architecture Behavioral of half_adder is
begin
    Sum <= A XOR B;
    Carry <= A AND B;
end Behavioral;

-- ========================================
-- 2. FULL ADDER
-- ========================================
entity full_adder is
    Port (
        A : in STD_LOGIC;
        B : in STD_LOGIC;
        Cin : in STD_LOGIC;
        Sum : out STD_LOGIC;
        Cout : out STD_LOGIC
    );
end full_adder;

architecture Behavioral of full_adder is
begin
    Sum <= A XOR B XOR Cin;
    Cout <= (A AND B) OR (Cin AND (A XOR B));
end Behavioral;
```

```vhdl
-- ======================================
-- 3. 4-BIT RIPPLE CARRY ADDER
-- ======================================
entity ripple_carry_adder_4bit is
    Port (
        A : in STD_LOGIC_VECTOR(3 downto 0);
        B : in STD_LOGIC_VECTOR(3 downto 0);
        Cin : in STD_LOGIC;
        Sum : out STD_LOGIC_VECTOR(3 downto 0);
        Cout : out STD_LOGIC
    );
end ripple_carry_adder_4bit;

architecture Structural of ripple_carry_adder_4bit is
    component full_adder is
        Port (
            A : in STD_LOGIC;
            B : in STD_LOGIC;
            Cin : in STD_LOGIC;
            Sum : out STD_LOGIC;
            Cout : out STD_LOGIC
        );
    end component;

    signal C : STD_LOGIC_VECTOR(3 downto 0);
begin
    FA0: full_adder port map (A(0), B(0), Cin, Sum(0), C(0));
    FA1: full_adder port map (A(1), B(1), C(0), Sum(1), C(1));
    FA2: full_adder port map (A(2), B(2), C(1), Sum(2), C(2));
    FA3: full_adder port map (A(3), B(3), C(2), Sum(3), C(3));

    Cout <= C(3);
end Structural;

-- ======================================
-- 4. 4:1 MULTIPLEXER
-- ======================================
entity mux_4to1 is
    Port (
        I0, I1, I2, I3 : in STD_LOGIC;
        S : in STD_LOGIC_VECTOR(1 downto 0);
        Y : out STD_LOGIC
    );
end mux_4to1;

architecture Behavioral of mux_4to1 is
```

```
begin
    with S select
        Y <= I0 when "00",
             I1 when "01",
             I2 when "10",
             I3 when "11",
             'X' when others;
end Behavioral;

-- Alternative implementation using if-else
architecture Behavioral_Alt of mux_4to1 is
begin
    process(I0, I1, I2, I3, S)
    begin
        if S = "00" then
            Y <= I0;
        elsif S = "01" then
            Y <= I1;
        elsif S = "10" then
            Y <= I2;
        elsif S = "11" then
            Y <= I3;
        else
            Y <= 'X';
        end if;
    end process;
end Behavioral_Alt;

-- ========================================
-- 5. 8:1 MULTIPLEXER
-- ========================================
entity mux_8to1 is
    Port (
        I : in STD_LOGIC_VECTOR(7 downto 0);
        S : in STD_LOGIC_VECTOR(2 downto 0);
        Y : out STD_LOGIC
    );
end mux_8to1;

architecture Behavioral of mux_8to1 is
begin
    with S select
        Y <= I(0) when "000",
             I(1) when "001",
             I(2) when "010",
             I(3) when "011",
```

```
            I(4) when "100",
            I(5) when "101",
            I(6) when "110",
            I(7) when "111",
            'X' when others;
end Behavioral;

-- ======================================
-- 6. 1:4 DEMULTIPLEXER
-- ======================================
entity demux_1to4 is
    Port (
        D : in STD_LOGIC;
        S : in STD_LOGIC_VECTOR(1 downto 0);
        Y : out STD_LOGIC_VECTOR(3 downto 0)
    );
end demux_1to4;

architecture Behavioral of demux_1to4 is
begin
    process(D, S)
    begin
        Y <= "0000";  -- Default all outputs to 0
        case S is
            when "00" => Y(0) <= D;
            when "01" => Y(1) <= D;
            when "10" => Y(2) <= D;
            when "11" => Y(3) <= D;
            when others => Y <= "0000";
        end case;
    end process;
end Behavioral;

-- ======================================
-- 7. 1:8 DEMULTIPLEXER
-- ======================================
entity demux_1to8 is
    Port (
        D : in STD_LOGIC;
        S : in STD_LOGIC_VECTOR(2 downto 0);
        Y : out STD_LOGIC_VECTOR(7 downto 0)
    );
end demux_1to8;

architecture Behavioral of demux_1to8 is
begin
```

```
    process(D, S)
    begin
        Y <= "00000000";  -- Default all outputs to 0
        case S is
            when "000" => Y(0) <= D;
            when "001" => Y(1) <= D;
            when "010" => Y(2) <= D;
            when "011" => Y(3) <= D;
            when "100" => Y(4) <= D;
            when "101" => Y(5) <= D;
            when "110" => Y(6) <= D;
            when "111" => Y(7) <= D;
            when others => Y <= "00000000";
        end case;
    end process;
end Behavioral;

-- ========================================
-- 8. 4:2 ENCODER (Priority Encoder)
-- ========================================
entity encoder_4to2 is
    Port (
        D : in STD_LOGIC_VECTOR(3 downto 0);
        Y : out STD_LOGIC_VECTOR(1 downto 0);
        Valid : out STD_LOGIC
    );
end encoder_4to2;

architecture Behavioral of encoder_4to2 is
begin
    process(D)
    begin
        if D(3) = '1' then
            Y <= "11";
            Valid <= '1';
        elsif D(2) = '1' then
            Y <= "10";
            Valid <= '1';
        elsif D(1) = '1' then
            Y <= "01";
            Valid <= '1';
        elsif D(0) = '1' then
            Y <= "00";
            Valid <= '1';
        else
            Y <= "00";
```

```
            Valid <= '0';
        end if;
    end process;
end Behavioral;

-- ======================================
-- 9. 8:3 ENCODER (Priority Encoder)
-- ======================================
entity encoder_8to3 is
    Port (
        D : in STD_LOGIC_VECTOR(7 downto 0);
        Y : out STD_LOGIC_VECTOR(2 downto 0);
        Valid : out STD_LOGIC
    );
end encoder_8to3;

architecture Behavioral of encoder_8to3 is
begin
    process(D)
    begin
        if D(7) = '1' then
            Y <= "111";
            Valid <= '1';
        elsif D(6) = '1' then
            Y <= "110";
            Valid <= '1';
        elsif D(5) = '1' then
            Y <= "101";
            Valid <= '1';
        elsif D(4) = '1' then
            Y <= "100";
            Valid <= '1';
        elsif D(3) = '1' then
            Y <= "011";
            Valid <= '1';
        elsif D(2) = '1' then
            Y <= "010";
            Valid <= '1';
        elsif D(1) = '1' then
            Y <= "001";
            Valid <= '1';
        elsif D(0) = '1' then
            Y <= "000";
            Valid <= '1';
        else
            Y <= "000";
```

```
            Valid <= '0';
        end if;
    end process;
end Behavioral;

-- ======================================
-- 10. 2:4 DECODER
-- ======================================
entity decoder_2to4 is
    Port (
        A : in STD_LOGIC_VECTOR(1 downto 0);
        Enable : in STD_LOGIC;
        Y : out STD_LOGIC_VECTOR(3 downto 0)
    );
end decoder_2to4;

architecture Behavioral of decoder_2to4 is
begin
    process(A, Enable)
    begin
        if Enable = '1' then
            case A is
                when "00" => Y <= "0001";
                when "01" => Y <= "0010";
                when "10" => Y <= "0100";
                when "11" => Y <= "1000";
                when others => Y <= "0000";
            end case;
        else
            Y <= "0000";
        end if;
    end process;
end Behavioral;

-- ======================================
-- 11. 3:8 DECODER
-- ======================================
entity decoder_3to8 is
    Port (
        A : in STD_LOGIC_VECTOR(2 downto 0);
        Enable : in STD_LOGIC;
        Y : out STD_LOGIC_VECTOR(7 downto 0)
    );
end decoder_3to8;

architecture Behavioral of decoder_3to8 is
```

```
begin
    process(A, Enable)
    begin
        if Enable = '1' then
            case A is
                when "000" => Y <= "00000001";
                when "001" => Y <= "00000010";
                when "010" => Y <= "00000100";
                when "011" => Y <= "00001000";
                when "100" => Y <= "00010000";
                when "101" => Y <= "00100000";
                when "110" => Y <= "01000000";
                when "111" => Y <= "10000000";
                when others => Y <= "00000000";
            end case;
        else
            Y <= "00000000";
        end if;
    end process;
end Behavioral;

-- ========================================
-- 12. 2-BIT MAGNITUDE COMPARATOR
-- ========================================
entity comparator_2bit is
    Port (
        A : in STD_LOGIC_VECTOR(1 downto 0);
        B : in STD_LOGIC_VECTOR(1 downto 0);
        AgtB : out STD_LOGIC;  -- A > B
        AeqB : out STD_LOGIC;  -- A = B
        AltB : out STD_LOGIC   -- A < B
    );
end comparator_2bit;

architecture Behavioral of comparator_2bit is
begin
    process(A, B)
    begin
        if unsigned(A) > unsigned(B) then
            AgtB <= '1';
            AeqB <= '0';
            AltB <= '0';
        elsif unsigned(A) = unsigned(B) then
            AgtB <= '0';
            AeqB <= '1';
            AltB <= '0';
```

```
        else
            AgtB <= '0';
            AeqB <= '0';
            AltB <= '1';
        end if;
    end process;
end Behavioral;

-- ======================================
-- 13. 4-BIT MAGNITUDE COMPARATOR
-- ======================================
entity comparator_4bit is
    Port (
        A : in STD_LOGIC_VECTOR(3 downto 0);
        B : in STD_LOGIC_VECTOR(3 downto 0);
        AgtB : out STD_LOGIC;  -- A > B
        AeqB : out STD_LOGIC;  -- A = B
        AltB : out STD_LOGIC   -- A < B
    );
end comparator_4bit;

architecture Behavioral of comparator_4bit is
begin
    process(A, B)
    begin
        if unsigned(A) > unsigned(B) then
            AgtB <= '1';
            AeqB <= '0';
            AltB <= '0';
        elsif unsigned(A) = unsigned(B) then
            AgtB <= '0';
            AeqB <= '1';
            AltB <= '0';
        else
            AgtB <= '0';
            AeqB <= '0';
            AltB <= '1';
        end if;
    end process;
end Behavioral;

-- ======================================
-- 14. BCD to 7-SEGMENT DECODER
-- ======================================
entity bcd_to_7seg is
    Port (
```

```
        BCD : in STD_LOGIC_VECTOR(3 downto 0);
        Seven_Seg : out STD_LOGIC_VECTOR(6 downto 0)  -- a,b,c,d,e,f,g
    );
end bcd_to_7seg;

architecture Behavioral of bcd_to_7seg is
begin
    process(BCD)
    begin
        case BCD is
            when "0000" => Seven_Seg <= "0000001";  -- 0
            when "0001" => Seven_Seg <= "1001111";  -- 1
            when "0010" => Seven_Seg <= "0010010";  -- 2
            when "0011" => Seven_Seg <= "0000110";  -- 3
            when "0100" => Seven_Seg <= "1001100";  -- 4
            when "0101" => Seven_Seg <= "0100100";  -- 5
            when "0110" => Seven_Seg <= "0100000";  -- 6
            when "0111" => Seven_Seg <= "0001111";  -- 7
            when "1000" => Seven_Seg <= "0000000";  -- 8
            when "1001" => Seven_Seg <= "0000100";  -- 9
            when others => Seven_Seg <= "1111111";  -- Blank for invalid BCD
        end case;
    end process;
end Behavioral;

-- ========================================
-- 15. 3-INPUT MAJORITY CIRCUIT
-- ========================================
entity majority_3input is
    Port (
        A, B, C : in STD_LOGIC;
        Y : out STD_LOGIC
    );
end majority_3input;

architecture Behavioral of majority_3input is
begin
    Y <= (A AND B) OR (A AND C) OR (B AND C);
end Behavioral;

-- ========================================
-- 16. PARITY GENERATOR (EVEN PARITY)
-- ========================================
entity parity_generator_4bit is
    Port (
        Data : in STD_LOGIC_VECTOR(3 downto 0);
```

```
        Parity : out STD_LOGIC
    );
end parity_generator_4bit;

architecture Behavioral of parity_generator_4bit is
begin
    Parity <= Data(0) XOR Data(1) XOR Data(2) XOR Data(3);
end Behavioral;

-- ========================================
-- 17. PARITY CHECKER (EVEN PARITY)
-- ========================================
entity parity_checker_4bit is
    Port (
        Data : in STD_LOGIC_VECTOR(3 downto 0);
        Parity_In : in STD_LOGIC;
        Error : out STD_LOGIC
    );
end parity_checker_4bit;

architecture Behavioral of parity_checker_4bit is
begin
    Error <= Data(0) XOR Data(1) XOR Data(2) XOR Data(3) XOR Parity_In;
end Behavioral;

-- ========================================
-- 18. BINARY TO GRAY CODE CONVERTER
-- ========================================
entity binary_to_gray_4bit is
    Port (
        Binary : in STD_LOGIC_VECTOR(3 downto 0);
        Gray : out STD_LOGIC_VECTOR(3 downto 0)
    );
end binary_to_gray_4bit;

architecture Behavioral of binary_to_gray_4bit is
begin
    Gray(3) <= Binary(3);
    Gray(2) <= Binary(3) XOR Binary(2);
    Gray(1) <= Binary(2) XOR Binary(1);
    Gray(0) <= Binary(1) XOR Binary(0);
end Behavioral;

-- ========================================
-- 19. GRAY TO BINARY CODE CONVERTER
-- ========================================
```

```
entity gray_to_binary_4bit is
    Port (
        Gray : in STD_LOGIC_VECTOR(3 downto 0);
        Binary : out STD_LOGIC_VECTOR(3 downto 0)
    );
end gray_to_binary_4bit;

architecture Behavioral of gray_to_binary_4bit is
begin
    Binary(3) <= Gray(3);
    Binary(2) <= Binary(3) XOR Gray(2);
    Binary(1) <= Binary(2) XOR Gray(1);
    Binary(0) <= Binary(1) XOR Gray(0);
end Behavioral;
```

This tutorial was generated with Claude AI(accessed: 2025-09-01). Modified and reviewed by JACH for CMSC 132.